感知流（Perception Stream）设计方案

一、背景与动机

当前系统中，音频采集、屏幕 OCR、主动 OCR、用户输入等感知能力**各自独立运行**，各有各的数据格式、回调方式和存储路径。下游消费者（主动服务、知识库、日志等）需要分别对接每条管线，导致：

- 下游模块与感知源**强耦合**，每加一个感知源就要改 N 个下游
- 没有统一的时间线视图，难以做**跨模态关联**（如"用户正在看微信时说了一句话"）
- 无法统一控制感知的开关、优先级和流量

感知流的目标：在所有感知源和所有下游消费者之间插入一条**统一的数据总线**， 所有感知事件归一化后流过这条总线，下游按需订阅。

二、感知源清单

#	感知源	数据类型	当前实现位置	产出频率
1	电脑麦克风	音频流 → ASR 文本	`audio_ws.py` / `audio_service`	实时流式
2	硬件麦克风（录音豆）	音频流 → ASR 文本	`hardware_audio.py`	实时流式
3	屏幕定时截图 + OCR	截图 + OCR 文本	`jobs/ocr.py` / `recorder.py`	每 N 秒一次
4	主动 OCR（微信/飞书）	结构化聊天记录	`jobs/proactive_ocr/`	事件驱动
5	用户手动输入	文本 / 指令	`chat` router / 悬浮窗	用户触发
6	（未来）摄像头 / 传感器	图像 / 信号	—	待定

三、核心概念

3.1 感知事件（PerceptionEvent）

所有感知源产出的数据，统一归一化为一个 **PerceptionEvent**：

```
PerceptionEvent
├── event_id        : str          # 唯一 ID（UUID）
├── sequence_id     : int          # 全局单调递增序列号，由 Stream 统一分配
├── timestamp       : datetime     # 事件产生时间（感知源本地时钟）
├── ingested_at     : datetime     # 事件进入 Stream 的时间（Stream 服务端时钟）
├── source          : SourceType   # 来源枚举：mic_pc / mic_hardware / ocr_screen /
ocr_proactive / user_input
├── modality        : Modality     # 模态枚举：audio / image / text
├── content_text    : str          # 文本化内容（所有模态最终都产出文本）
├── content_raw     : Any | None   # 可选的原始内容（截图路径、音频片段 ID 等）
├── metadata        : dict         # 元信息（说话人、来源 APP、窗口标题、情绪标签等）
└── priority        : int          # 优先级（0=低，用于下游过滤）
```

排序机制：

- `sequence_id` 是**全局唯一且单调递增**的，由 Stream 在 `publish()` 时统一分配（原子自增计数器）
- 下游消费者以 `sequence_id` 作为**事件的权威顺序**，而非 `timestamp`
- `timestamp` 仅作为参考（反映感知源本地时钟，不同源之间可能有偏移）
- `ingested_at` 记录事件进入 Stream 的统一时钟时间，用于跨模态时间对齐和调试

核心原则：**所有模态最终都转为文本**（text-first），`content_text` 是必填字段。 原始数据通过 `content_raw` 保留引用，不在流中传输大二进制。

3.2 感知流（PerceptionStream）

感知流是一个**进程内的发布-订阅总线**：

```
┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
│ 电脑麦克风 │  │ 硬件麦克风 │  │ 屏幕 OCR  │  │ 主动 OCR  │  │ 用户输入  │
└──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘
      │             │             │             │             │
      │   produce   │   produce   │   produce   │   produce   │
      ▼             ▼             ▼             ▼             ▼
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
```

```
│                    Perception Stream（感知总线）                    │
│                                                                    │
│    ┌──────────────────────────────────────────────────────────┐    │
│    │  内部：有序事件队列（按 sequence_id 排序）+ 短期滑动窗口缓存 │    │
│    └──────────────────────────────────────────────────────────┘    │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
        │          │          │          │          │
        ▼          ▼          ▼          ▼          ▼
    ┌─────────┐ ┌─────────┐ ┌─────────┐ ┌─────────┐ ┌─────────┐
    │主动服务 │ │知识库   │ │ 日志    │ │上下文   │ │ ...     │
    │(Agent)  │ │写入     │ │记录     │ │构建器   │ │扩展     │
    └─────────┘ └─────────┘ └─────────┘ └─────────┘ └─────────┘

      subscribe   subscribe   subscribe   subscribe
```

3.3 适配器（Source Adapter）

每个感知源配一个**适配器**，职责：

1. 监听原有模块的数据产出（回调 / 队列 / 事件）
2. 将原始数据转为 `PerceptionEvent`
3. 调用 `stream.publish(event)` 推入总线

适配器是**薄胶水层**，不改变现有感知模块的内部逻辑。

四、整体架构

```
                          ┌──────────────────────────┐
                          │     Config（config.yaml） │
                          │  - 各感知源开关           │
                          │  - 采集频率               │
                          │  - 优先级规则             │
                          └──────────────────────────┘
                                      │ 读取配置
                                      ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      PerceptionStreamManager                         │
│                                                                      │
│  职责：                                                              │
│  1. 根据配置初始化/启停各感知源适配器                                  │
│  2. 管理 PerceptionStream 实例                                       │
│  3. 提供注册订阅者的 API                                             │
│  4. 暴露健康状态（哪些感知源在线）                                     │
│                                                                      │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐              │
│   │ AudioAdapter │  │  OCRAdapter  │  │ InputAdapter │  ...         │
```

```
│  │ (mic_pc +     │  │ (screen +     │  │ (chat +      │            │
│  │  mic_hw)      │  │  proactive)   │  │  floating)   │            │
│  └──────┬────────┘  └──────┬────────┘  └──────┬───────┘            │
│         └─────────────────┼─────────────────┘                      │
│                           ▼                                        │
│                   PerceptionStream                                 │
│                 （发布-订阅 + 滑动窗口）                            │
│                           │                                        │
│                ┌──────────┼──────────┐                             │
│                ▼          ▼          ▼                             │
│          Subscriber   Subscriber   Subscriber                      │
└────────────────────────────────────────────────────────────────────┘
```

五、数据流转流程

以"用户在微信收到一条消息，同时说了一句话"为例：

```
时间轴 ──────────────────────────────────────────────────────────►

T1: 主动 OCR 检测到微信新消息
    │
    ├─ ProactiveOCRAdapter 生成 PerceptionEvent:
    │   source=ocr_proactive, content_text="张三: 下午3点能开会吗？"
    │   metadata={app: "wechat", sender: "张三", msg_type: "question"}
    │
    └─ publish → Stream

T2: 电脑麦克风捕获用户语音 "嗯...下午三点好像有课"
    │
    ├─ AudioAdapter（等 ASR 完成后）生成 PerceptionEvent:
    │   source=mic_pc, content_text="嗯...下午三点好像有课"
    │   metadata={speaker: "self", confidence: 0.92}
    │
    └─ publish → Stream

T3: 下游订阅者收到两个事件
    │
    ├─ 主动服务 Agent：关联两个事件 → 识别出"用户被问能否开会，自己在犹豫"
    │   → 触发大模型：查课表 + 生成建议回复
    │
    ├─ 知识库写入：两条记录入库，携带时间戳和元信息
    │
    └─ 日志：记录事件流水
```

六、关键设计决策

6.1 进程内 vs 消息队列

方案	优点	缺点
进程内 asyncio 队列（推荐 MVP）	零部署成本，延迟极低	单进程，无持久化
Redis Stream / Kafka	持久化，跨进程	引入外部依赖，过度设计

决策：MVP 阶段用 `asyncio.Queue` + 内存滑动窗口，够用就行。 未来如果需要多进程或持久化，再切换到 Redis Stream。

6.2 滑动窗口

Stream 内部维护一个**时间窗口**（默认 5 分钟），用于：

- 下游订阅者启动时可以回放近期事件（不丢失启动前几秒的数据）
- 跨模态关联时提供短期上下文
- 窗口之外的事件自动丢弃（长期存储由知识库订阅者负责）

6.3 背压与优先级

- 如果下游消费者处理不过来，事件在队列中堆积超过阈值时，**丢弃低优先级事件**
- 用户手动输入 > 音频实时转录 > 主动 OCR > 定时截图 OCR

6.4 与现有模块的关系

不侵入现有代码。适配器通过以下方式接入：

- 音频：在现有 ASR 回调完成后，适配器拿到文本，生成事件
- OCR：在现有 OCR Job 完成一轮后，适配器拿到结果，生成事件
- 用户输入：在 chat router 收到消息时，适配器拷贝一份，生成事件

现有模块继续独立工作，感知流是**旁路叠加**，不影响原有功能。

七、模块划分

```
lifetrace/
├── perception/                    # 新增：感知流模块
│   ├── __init__.py
│   ├── stream.py                  # PerceptionStream 核心（发布-订阅 + 滑动窗口）
│   ├── models.py                  # PerceptionEvent / SourceType / Modality 数据模
型
│   ├── manager.py                 # PerceptionStreamManager（生命周期管理）
│   ├── adapters/                  # 感知源适配器
│   │   ├── __init__.py
│   │   ├── audio_adapter.py       # 电脑麦克风 + 硬件麦克风 → Event
│   │   ├── ocr_adapter.py         # 屏幕 OCR + 主动 OCR → Event
│   │   └── input_adapter.py       # 用户手动输入 → Event
│   └── subscribers/               # 内置订阅者（可选）
│       ├── __init__.py
│       ├── logger_subscriber.py   # 感知事件日志
│       └── context_subscriber.py  # 为 Agent 构建实时上下文窗口
```

十、开放问题

1. **事件去重**：同一条微信消息可能被主动 OCR 多次捕获，如何去重？ → 初步方案：基于 content_text 哈希 + 时间窗口去重
2. **多模态对齐**：音频事件和 OCR 事件的时间戳可能有偏移，如何对齐？ → 初步方案：允许 ±3 秒的模糊匹配窗口
3. **隐私控制**：用户是否需要细粒度控制哪些感知源的数据进入流？ → 配置文件 + 前端开关
4. **持久化时机**：Stream 本身不持久化，但是否需要一个"存档订阅者"把全量事件落盘？ → Phase 2 之后根据需要决定